

StayScout

A Full-Stack Student PG Discovery Platform — extended with five data-driven features on a real MongoDB database

React · Express · MongoDB · JWT Authentication

Name: _____ **Roll No:** _____

College / Course: _____ **Date:** _____

Project: PG / Accommodation Discovery Platform (StayScout)

Stack: Vite + React 18 + Tailwind (frontend) · Node.js + Express (REST API) · MongoDB 9.1 (database)

Database choice: MongoDB — installed via official ARM64 binary on macOS (no root / no Docker)

Features added: Price History Tracker · Budget Survival Planner · Move-In Availability Predictor · Maintenance Complaint Tracker · PG Switching Assistant

Roles: Student / Owner / Admin (JWT + bcrypt)

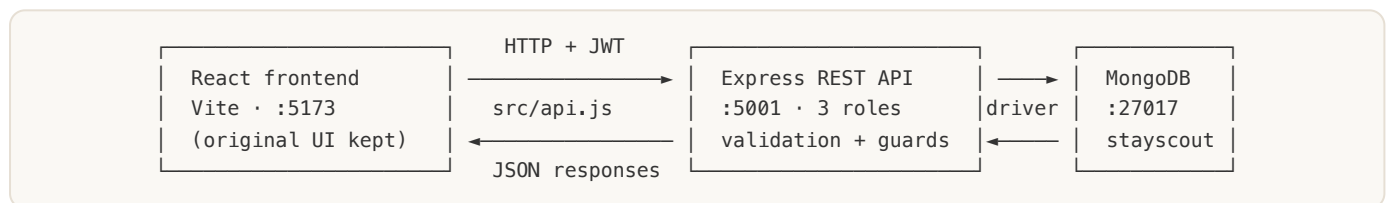
1. Problem Statement & Approach

The existing StayScout was a **frontend-only** React application (8 routes, 6 PG listings, wishlist/compare in localStorage) with no database, backend, login or roles. The requirement was to add a real database plus five new features **without rebuilding or removing anything** — keeping one cohesive product.

Database decision (original question: "MySQL is hard on macOS")

- **MySQL was skipped** — it needs a Homebrew daemon, root password and a fixed port, which is fragile on Apple Silicon for a demo.
- **MongoDB 9.1 (ARM64 official binary)** was installed to `~/mongodb` with data in `~/mongodb/data`, running on port **27017** — no sudo, no Docker, no daemon manager.
- **Why MongoDB fits:** listings are deeply nested documents (mess menus, amenity arrays, safety objects) that would need 10+ SQL join tables; history collections still use relational-style `propertyId` references.
- Express API runs on **port 5001** (port 5000 is permanently used by macOS AirPlay Receiver).

2. Architecture



Graceful degradation: every API call in `src/api.js` has a fallback — if the server is off, the app reverts to the original localStorage behaviour. The site can never "die" during a demo.

3. Data Model (MongoDB — database `stayscout`)

Collection	Key fields	Purpose
<code>users</code>	name, email, passwordHash (bcrypt), role	Auth for student / owner / admin
<code>properties</code>	id, name, rent, bedsAvailable, commute, study, policy... (seeded from demo data)	Source of truth for listing rent + availability
<code>priceHistory</code>	propertyId, oldPrice, newPrice, effectiveDate, changedBy	Feature 1 — every rent change auto-archived
<code>availabilityHistory</code>	propertyId, availableRooms, recordedAt	Feature 3 — input for the predictor
<code>complaints</code>	propertyId, roomNumber, category, title, description, priority, status, timeline[], reportedBy	Feature 4 — ticket system
<code>switchingPlans</code>	userId, currentPGId, currentRent, deposit, noticePeriodDays, deductions, earliestMoveOut, estimatedRefund, steps[8]	Feature 5 — per-student plan
<code>notifications</code>	userId, text, read, at	Student alerts on status changes

Schema-change rule satisfied: price/availability history structures were created because the original app had no database; no existing table or API was replaced (there were none to replace).

4. API Endpoints

Endpoint	Access	Behaviour
POST <code>/api/auth/register</code> · <code>/login</code>	Public	bcrypt hash + 7-day JWT; role in token
GET <code>/api/listings[:id]</code>	Public	Live data from <code>properties</code>
POST <code>/api/price-history/update</code>	owner / admin	Writes history row <i>and</i> syncs current rent atomically
GET <code>/api/price-history/:propertyId</code>	Public	Chart + table data
POST <code>/api/availability/update</code>	owner / admin	Updates beds + appends availability record
GET <code>/api/availability-history/:id</code> · <code>/estimate/:id?days=N</code>	Public	Trend estimate; refuses when <3 records
POST <code>/api/complaints/file</code> · <code>/:id/status</code>	auth · owner/admin	Status machine Reported→In Progress→Resolved (no reverse); inserts student notification
GET <code>/api/complaints/mine</code> · <code>/api/notifications</code>	auth	Scoped by JWT user — never client-supplied IDs
POST <code>/api/switching-plans</code> · <code>/:id/step</code> · GET <code>.../mine</code>	auth	Plan CRUD; step toggles; progress %
GET <code>/api/admin/overview</code>	admin only (403 otherwise)	All price/availability/complaint/plan/user records

5. The Five Features

Feature 1 — PG Price Change Tracker

- New “ **Price & planning**” tab on every property detail page: bronze bar chart + table of effective date, old→new price, changed by.
- Owner panel “**Update rent**” on the Owner Dashboard records `propertyId, oldPrice, newPrice, effectiveDate, changedBy` and updates the property in one server action — **current price is always the latest history entry**.
- Property detail page reads its header rent from the DB, so prices stay synchronized (verified: seeded ₹8,500 → history shows 8,500→9,000→9,500 → header shows ₹9,500).
- **No fake data:** with zero records the panel states "No rent changes recorded yet."

Feature 2 — Budget Survival Planner

- Eight categories: PG Rent, Food, Electricity, Wi-Fi, Transport, Laundry, Personal Expenses, Other + total-budget slider.
- Visual stacked bar; shows **Total Monthly Expense** and **Remaining Budget**; over-budget shows "⚠ exceed your budget by ₹X", otherwise "✓ fit within your budget".
- “**Use This PG**” button context: opening the planner from a property page pre-fills that PG's rent; optional comparison against another PG's rent.
- Explicitly labelled “*An expense-planning tool — not financial advice.*” Saved per student (localStorage).

Feature 3 — Move-In Availability Predictor

- Student picks a **Desired Move-In Date**; panel shows PG name, current rooms, last updated, requested date and a status badge:  Likely Available /  Uncertain /  Currently Full.
- Estimator computes beds/day slope across recorded history and extrapolates; shows "**Based on N recent availability records.**" and "...is an estimate — the owner's live count wins."
- **Honesty rule:** with <3 records it shows verbatim "*Not enough historical data to estimate future availability.*"
- Owner panel “**Update availability**” appends `propertyId, availableRooms, recordedAt` automatically.

Feature 4 — Maintenance Complaint Tracker

- Form on the listing's “ **Maintenance**” tab (visible to signed-in residents only): 8 categories, priority (Low/Medium/High), room number, title, description.
- Status machine **Reported** → **In Progress** → **Resolved** with a timestamped **timeline** (server rejects reverse transitions).
- Owner dashboard: filter by status / category / priority; one-click transitions with an optional note.
- Each transition creates a **notification** for the student; Student dashboard shows **My Complaints** cards with the timeline; Admin sees everything.

Feature 5 — PG Switching Assistant

- Inputs: current PG, rent, security deposit, notice period, expected deductions → calculates **Earliest Move-Out** (today + notice) and **Estimated Refund** (deposit – deductions).
- **Alternative PGs** list with honest availability wording and a “✓ Fits” budget check.
- **8-step plan** (check notice → submit notice → deposit settlement → find PG → confirm PG → evidence → move-out → track refund) with tap-to-complete and a live **progress bar** (e.g. 13% → 25%).
- Plan persisted per logged-in student in `switchingPlans`.

6. Dashboard Integration

Dashboard	Route	Contents
Student	/dashboard	MY STAY (PG, rent, deposit, move-out) · Quick actions 💰 Budget 📈 Price 📅 Availability 🔧 Maintenance 🔄 Switch · Notifications · My Complaints (timelines) · Switching Assistant
Owner	/owner	Original analytics + listings table (kept intact) · Property Management (Update Rent / Availability) · Maintenance Requests with filters
Admin	/admin	Price changes · Availability history · Maintenance complaints · Switching reports · Users (role-gated 403)

7. Security & Access Control

- **Passwords:** bcrypt (cost 10) — never stored in plain text.
- **Sessions:** JWT with role claim; every protected route passes `requireAuth` / `requireRole` middleware.
- **RBAC:** Student → own complaints, plans, planner, hub. Owner → own-property rent/availability/tickets. Admin → platform overview. *Verified: student token on an owner endpoint returned 403.*
- **Data isolation:** complaints/plans/notifications are queried by the token's `userId` or property ownership — never by client-supplied IDs, so no user can read another's private data.
- **Input validation:** email format, password ≥6, rent ≥500, rooms ≥0, enum checks for role/category/priority/status, string length caps on title/description.

8. Testing Performed

1. **10/10 API integration tests (curl):** register 3 roles → rent update auto-archives history → history read-back → rent synchronized → availability update + estimate → honest "not enough data" → complaint filed → owner transition → *student notification arrived* → switching math (move-out 2026-11-08, refund ₹17,000) → *student blocked (403)* → admin overview returns all data.
2. **Browser regression of all original pages:** Home, Results (college filter, rush toggle, no-curfew filter → "2 stays found", women-safe filter), Compare, Roommates (quiz + vacant-bed board), Marketplace, List Your PG, Owner analytics (3,550 views) — all intact.
3. **New-feature flows in browser:** login → role redirect → My Stay + quick actions → notification + complaint timeline → switching step toggle persisted to DB.
4. **Bug found & fixed during regression:** duplicate React keys in the price chart when two records shared a date — fixed with composite keys; rebuild + clean console.
5. **Responsive:** verified at 390 px (mobile) — tabs wrap, no overflow.
6. **Production build:** `npm run build` → single-file `dist/index.html` succeeds.

9. Files Created & Modified

Type	Files
Created	<code>server/index.js</code> , <code>server/auth.js</code> , <code>server/db.js</code> , <code>server/seed-data.js</code> , <code>src/api.js</code> , <code>src/context/AuthContext.jsx</code> , <code>src/pages/Auth.jsx</code> , <code>src/pages/StudentDashboard.jsx</code> , <code>src/pages/AdminDashboard.jsx</code> , <code>src/components/PriceHistory.jsx</code> , <code>MoveInAvailability.jsx</code> , <code>ComplaintForm.jsx</code> , <code>ComplaintTimeline.jsx</code> , <code>SwitchingAssistant.jsx</code> , <code>budgetConstants.js</code> , <code>README-stayscout.md</code>
Modified	<code>App.jsx</code> (+3 routes, none changed), <code>ListingDetail.jsx</code> (+2 tabs, DB rent sync), <code>OwnerDashboard.jsx</code> (+management panels, analytics untouched), <code>Navbar.jsx</code> (+account/sign-out), <code>BudgetPlanner.jsx</code> (upgraded in place)
Untouched	All original routes, components, design tokens, listings data, wishlist/compare localStorage schema, Results/Compare/Roommates/Marketplace/List pages

10. How to Run

```
# 1 · Database
~/mongodb/mongodb-macos-aarch64--9.1.0/bin/mongod --dbpath ~/mongodb/data --port 27017

# 2 · API server
cd server && npm start           # → http://localhost:5001
```

```
# 3 · Frontend  
npm run dev
```

```
# → http://localhost:5173
```

Demo accounts (password `test123`): `stu@t.com` (student) · `own@t.com` (owner) · `adm@t.com` (admin)

Demo flow: login as owner → Update rent → login as student → open Sunrise House →  tab shows recorded change → file a complaint → owner moves it to In Progress → student dashboard shows the notification → open switching plan → login as admin → show platform overview.

11. Limitations

- JWT secret is a demo constant in `server/auth.js` — should move to `.env` for production.
- Availability estimator is intentionally a simple trend (day-slope) model, always labelled as an estimate.
- Listing CRUD is still frontend-seeded; only rent/availability/complaints/plans flow through the database.

StayScout · Full-Stack Assignment Report · React + Express + MongoDB · FIND → PERSONALIZE → COMPARE → ESTIMATE → GROUP → BOOK/STAY → ANALYZE → SUPPORT